Лекция 2.
Динамические библиотеки

1 Зачем?

Команда `ldd` для статически слинкованных файлов (флаг -static при линковке) пишет “not a dynamic executable”. Для нас это значит, что они запустятся независимо от того, есть ли у нашего пользователя те библиотеки, которые мы используем. Но при такой сборке даже простой hello world весит около 2 мегабайт. Это больно, особенно когда хочется иметь в системе много маленьких программ на все случаи жизни. Вот эту проблему и решают динамические библиотеки. Они уменьшают объём исполняемого файла,

Посмотрим, например, утилиту ls:
`which ls` покажет нам, где она находится,
`readelf -h <path>` сообщит, что она слинкована динамически,
`ldd <path>` выведет её зависимости.

Прикол в том, что эти зависимости загружаются в оперативную память один раз для всех, кому они нужныб и все запущенные программы просто переиспользуют один и тот же инстанс в памяти (точнее, переиспользуются только read-only секции, но об этом в следующих лекциях).

А теперь мы, допустим, мы захотели создать свою библиотеку. Создадим два файла:

`multiply.cpp`, наша библиотека:

```
int multiply(int x, int y) {
  return x * y;
}
```

`main.cpp`, который будет ее использовать:

```
#include <iostream>

int multiply(int, int);

int main() {
  int x, y;
  std::cin >> x >> y;
  std::cout << multiply(x, y);
}
```

Для начала соберём объектные файлы, а затем слинкуем их:

```
g++ -c main.cpp
g++ -c multiply.cpp
g++ main.o multiply.o
```

И оно даже заработает. Но сейчас мы просто статически прилинковали multiply.o к main.o, а мы хотели именно динамическую библиотеку.

1.1 Создание динамической библиотеки

Соберём файл с расширением `.so`:

```
  g++ -shared multiply.cpp -o multiply.so
```

Забавно, что у полученного файла может быть выставлен бит исполнения (`ls -l multiply.so`), но попытка запустить его как обычную программу заканчивается ошибкой segmentation fault. Посмотрим на файл подробнее:

```
readelf -a multiply.so
```

В заголовке он обозначен как динамическая библиотека:

```
Type: DYN (Shared object file)
```

В файле есть таблица динамических символов `.dynsym`. В ней находятся символы, которые может использовать динамический линковщик при запуске программы. Есть ещё похожая таблица `.symtab`, в ней содержатся все символы (и глобальные, и локальные), нужна она в основном для отладки и может быть удалена утилитой `strip`.

Утилита `objcopy` позволяет редактировать ELF-файлы. Ещё для работы с ними есть полезная утилита `patchelf`.

Несмотря на то, что код динамической библиотеки загружается в оперативную память один раз для нескольких процессов, секции `.data` и `.bss` у каждого процесса свои. Их страницы могут физически быть общими до первой записи из-за оптимизации *copy-on-write* (COW).

1.2 Линковка с библиотекой

Ключ `-l<name>` добавляет библиотеку при линковке:

```
g++ main.cpp -lmultiply
```

Пока не работает: линковщик ожидает, что имя файла библиотеки начинается с префикса `lib`, поэтому переименуем файл:

```
mv multiply.so libmultiply.so
```

Всё ещё не работает: линковщик ищет библиотеки только в определённых каталогах, в том числе в /usr/lib. Можно скопировать библиотеку туда:

```
sudo cp libmultiply.so /usr/lib
g++ main.cpp -lmultiply
```

Теперь сборка проходит, а зависимости программы можно посмотреть с помощью ldd. Однако копировать тестовую библиотеку в системный каталог необязательно. Удалим её оттуда и добавим текущий каталог в путь поиска линковщика ключом -L:

```
sudo rm /usr/lib/libmultiply.so
g++ main.cpp -lmultiply -L.
```

Сборка теперь пройдет, но программа при запуске может не найти библиотеку. Путь поиска на этом этапе можно задать переменной окружения LD_LIBRARY_PATH, но на некоторых системах он уже задан как /usr/local/lib:, а пустой элемент после двоеточия означает текущую папку, поэтому, вероятно, программа запустится и так. Иначе, можно явно указать директорию поиска вот так:

```
LD_LIBRARY_PATH=. ./a.out
```

Другой вариант — записать путь поиска в сам исполняемый файл при линковке:

```
g++ main.cpp -lmultiply -L. -Wl,-rpath=<absolute path>
g++ main.cpp -lmultiply -L. -Wl,-rpath=.
readelf -d a.out
```

В выводе readelf появится запись RUNPATH:

```
0x000000000000001d (RUNPATH) Library runpath: [.]
```

Итак, ключ -L задаёт путь поиска библиотек во время сборки, а LD_LIBRARY_PATH и RUNPATH — во время запуска.

1.3 SONAME

В заголовке динамической библиотеки может быть указано поле SONAME. Пусть при линковке используется файл libmultiply.so, а в его поле SONAME записано другое имя, например, libmultiplyhaha.so. Тогда линковщик найдёт библиотеку по имени файла, но в список зависимостей запишет именно SONAME.

Это позволяет при сборке использовать конкретную версию библиотеки, например libfoo.so.1.4.7, а при запуске искать её по символической ссылке вроде libfoo.so.1. Библиотеку можно обновлять, пока её интерфейс остаётся совместимым.

Зададим нашей библиотеке другой SONAME и ещё раз соберём программу:

```
g++ -shared multiply.cpp -o libmultiply.so -Wl,-soname=libmultiplyhaha.so
g++ main.cpp -lmultiply -L. -Wl,-rpath=.
ldd a.out
```

В результате в списке зависимостей окажется новое имя, но файла с таким именем ещё нет:

```
linux-vdso.so.1
libmultiplyhaha.so => not found
libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6
/lib64/ld-linux-x86-64.so.2
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1
```

После этого пересоберем библиотеку без -soname

```
g++ -shared multiply.cpp -o libmultiply.so
```

1.4 Patchelf

Путь поиска можно добавить и в уже собранный ELF-файл. Сначала соберём программу без RUNPATH, а затем изменим его утилитой patchelf:

```
g++ main.cpp -lmultiply -L.
patchelf a.out --set-rpath .
```

После этого программа запускается. Врзможно, patchelf у вас не установлен, на Ubuntu ставим так:

```
sudo apt install patchelf
```

2 Динамический линковщик

Возьмем код из предыдущей лекции:

```
#include <iostream>

int main() {
  int x;
  std::cin >> x;
  std::cout << x + 5;
  return 0;
}
```

Соберем объектный файл, а затем попробуем его слинковать. Чтобы было от чего отталкиваться, посмотрим зависимости обычного `a.out`:

```
ldd a.out
```

```
linux-vdso.so.1
libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6
/lib64/ld-linux-x86-64.so.2
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1
```

Попробуем вызвать `ld` вручную и перечислить ему нужные библиотеки:

```
ld main.o /lib/x86_64-linux-gnu/libstdc++.so.6 \
  /lib/x86_64-linux-gnu/libc.so.6 \
  /lib/x86_64-linux-gnu/libm.so.6 \
  /lib64/ld-linux-x86-64.so.2 \
  /lib/x86_64-linux-gnu/libgcc_s.so.1
```

Получим warning

```
ld: warning: cannot find entry symbol _start; defaulting to
```

А при запуске ошибку

```
bash: ./a.out: cannot execute: required file not found
```

Сначала разберёмся с warningом. Добавим объектные файлы, с которых начинается и которыми заканчивается наша программа:

```
ld main.o \
  /lib/x86_64-linux-gnu/libstdc++.so.6 \
  /lib/x86_64-linux-gnu/libm.so.6 \
  /lib64/ld-linux-x86-64.so.2 \
  /lib/x86_64-linux-gnu/libgcc_s.so.1 \
  /lib/x86_64-linux-gnu/libc.so.6 \
  /usr/lib/gcc/x86_64-linux-gnu/14/crtendS.o \
  /usr/lib/x86_64-linux-gnu/crtn.o \
  /usr/lib/x86_64-linux-gnu/Scrt1.o \
  /usr/lib/x86_64-linux-gnu/crti.o \
  /usr/lib/gcc/x86_64-linux-gnu/14/crtbeginS.o
```

Теперь она даже соберется, но тут всё ещё есть две ошибки. При ручной линковке можно получить другой путь к динамическому линковщику, из-за чего при запуске возникнет ошибка `No such file or directory` или упомянутая ранее `bash: ./a.out: cannot execute: required file not found`. Путь к интерпретатору можно посмотреть двумя способами:

```
patchelf --print-interpreter <file>
readelf -l <file>
```

А явно задать его линковщику можно ключом:

```
-dynamic-linker /lib64/ld-linux-x86-64.so.2
```

Попробуем:

```
ld main.o -dynamic-linker /lib64/ld-linux-x86-64.so.2 \
  /lib/x86_64-linux-gnu/libstdc++.so.6 \
  /lib/x86_64-linux-gnu/libm.so.6 \
  /lib64/ld-linux-x86-64.so.2 \
  /lib/x86_64-linux-gnu/libgcc_s.so.1 \
  /lib/x86_64-linux-gnu/libc.so.6 \
  /usr/lib/gcc/x86_64-linux-gnu/14/crtendS.o \
  /usr/lib/x86_64-linux-gnu/crtn.o \
  /usr/lib/x86_64-linux-gnu/Scrt1.o \
  /usr/lib/x86_64-linux-gnu/crti.o \
  /usr/lib/gcc/x86_64-linux-gnu/14/crtbeginS.o
```

...

И с высокой вероятностью получим segfault. Вторая наша ошибка была в том, что мы указали файлы не в том порядке. Попробуем теперь так:

```
ld -dynamic-linker /lib64/ld-linux-x86-64.so.2 \
  /usr/lib/x86_64-linux-gnu/Scrt1.o \
  /usr/lib/x86_64-linux-gnu/crti.o \
  /usr/lib/gcc/x86_64-linux-gnu/14/crtbeginS.o \
  main.o \
  /lib/x86_64-linux-gnu/libstdc++.so.6 \
  /lib/x86_64-linux-gnu/libm.so.6 \
  /lib64/ld-linux-x86-64.so.2 \
  /lib/x86_64-linux-gnu/libgcc_s.so.1 \
  /lib/x86_64-linux-gnu/libc.so.6 \
  /usr/lib/gcc/x86_64-linux-gnu/14/crtendS.o \
  /usr/lib/x86_64-linux-gnu/crtn.o
```

Иии... Всё заработает.

Отдельный интересный вопрос — как запускается сам динамический линковщик, который тоже является `.so`-файлом, но при этом должен запустить нашу программу. Прикол в том, что у `/lib64/ld-linux-x86-64.so.2`, и у `/lib/x86_64-linux-gnu/libc.so.6` есть точка входа: их можно использовать как библиотеки, а можно запускать напрямую.

3 Секции ELF и память

Подробную информацию о секциях можно получить командой:

```
readelf -t a.out
```

Основные секции:

- `.text` — машинный код;
- `.data` — изменяемые инициализированные глобальные и статические данные;
- `.rodata` — неизменяемые инициализированные глобальные и статические данные;
- `.bss` — неинициализированные или инициализированные нулём глобальные и статические данные (*Block Started by Symbol*).

Если привести указатель на изначально константный объект к неконстантному типу и попытаться выполнить запись, поведение программы будет неопределённым. Для глобального или статического объекта, попавшего в доступную только для чтения память, на практике это обычно приведёт к segmentation fault. Константная локальная переменная на стеке обычно не вызовет такой ошибки, но запись в неё всё равно остаётся неопределённым поведением.

Команда `size <file>` выводит размеры основных секций бинарного файла.

P.S. Часть примеров здесь следует собирать как PIE, но разбор этой темы слишком объемный для нашей лекции, а примеры будут работать и так, т.к. они довольно простые. Желающим разобраться предлагается воспользоваться гуглом и гпт.